

Mount Kenya University



Unlocking Infinite Possibilities

UNIT TITLE: ADVANCED CRYPTOGRAPHY

STUDENT NAME: Gakere Samuel

ADMISSION NUMBER: BSCCS/2024/55278

COURSE: B.S.C COMPUTER SCIENCE

LECTURE: MR NYORO MICHAEL

SUBMISSION DATE: 6/8/2026

Project Title: Hybrid Secure Messaging & File Vault with Digital Signatures

•Selected Technologies:

- Python 3.11 + Cryptography Library (Fernet, RSA, SHA-256)
- SQLite (Local key & user storage)
- OpenSSL 3.0 (Certificate generation)

Table of Contents

ADMISSION NUMBER: BSCCS/2024/55278.....	1
COURSE: B.S.C COMPUTER SCIENCE.....	1
.....	4
Week 1: Creation of The Virtual Enviroment & Required Tools Installation.....	4
Fig 1.1: Openssl version.....	4
Fig 1.2: Virtual environment creation.....	4
Fig 1.3: Activating environment (notice prompt change).....	5
Fig 1.4: Successful pip install.....	5
Fig 1.5: Complete environment test script.....	6
Week 2: Classical Cryptography and Cipher Design.....	6
Fig 2.1: Caesar Cipher implementation showing encryption of "CRYPTOGRAPHY"	6
Fig 2.2: Vigenère ("KALI"), Caesar Brute Force (Shift 3), and Input Validation.....	7
Fig 2.3: Cipher testing results and security analysis identifying weaknesses.....	7
Week 3: Stream Ciphers and Randomness Testing.....	9
Fig 3: LFSR Implementation, RC4 Stream Cipher, and Performance Results.....	9
Week 4: AES Block Cipher.....	9
Fig 4: AES-256-CBC Key Generation, Encryption/Decryption, Performance Test, and Security Features.....	9
Week 5: RSA Public Key Cryptography.....	10
Fig 5: RSA-2048 Key Pair Generation, Public Key Encryption, Private Key Decryption, and Security Comparison.....	10
Week 6: Hashing and Password Security.....	10
Fig 6: SHA-256 Hash Generation, PBKDF2 Password Hashing with Salt, Login Authentication, and Security Analysis.....	10
Creativity.....	11

Fig 1.1: Openssl version

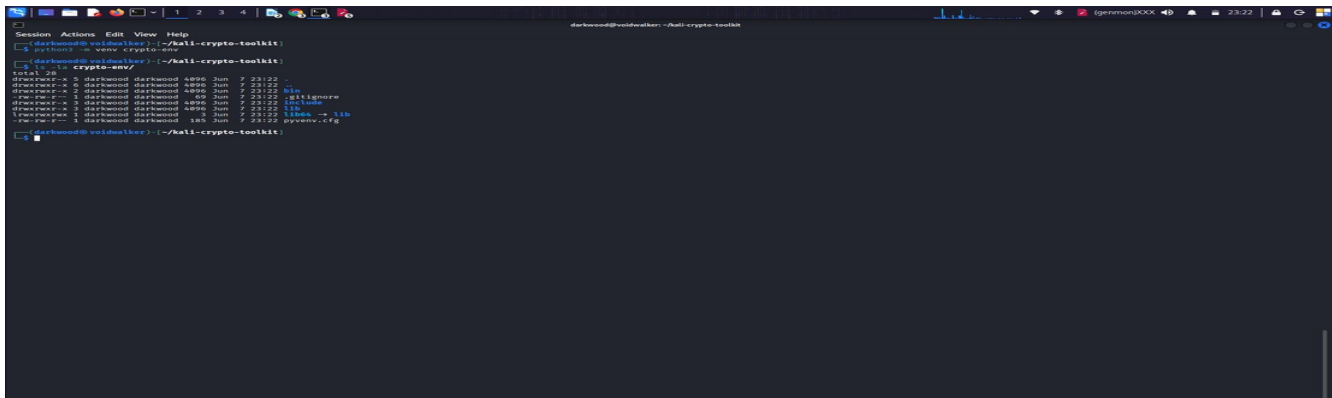
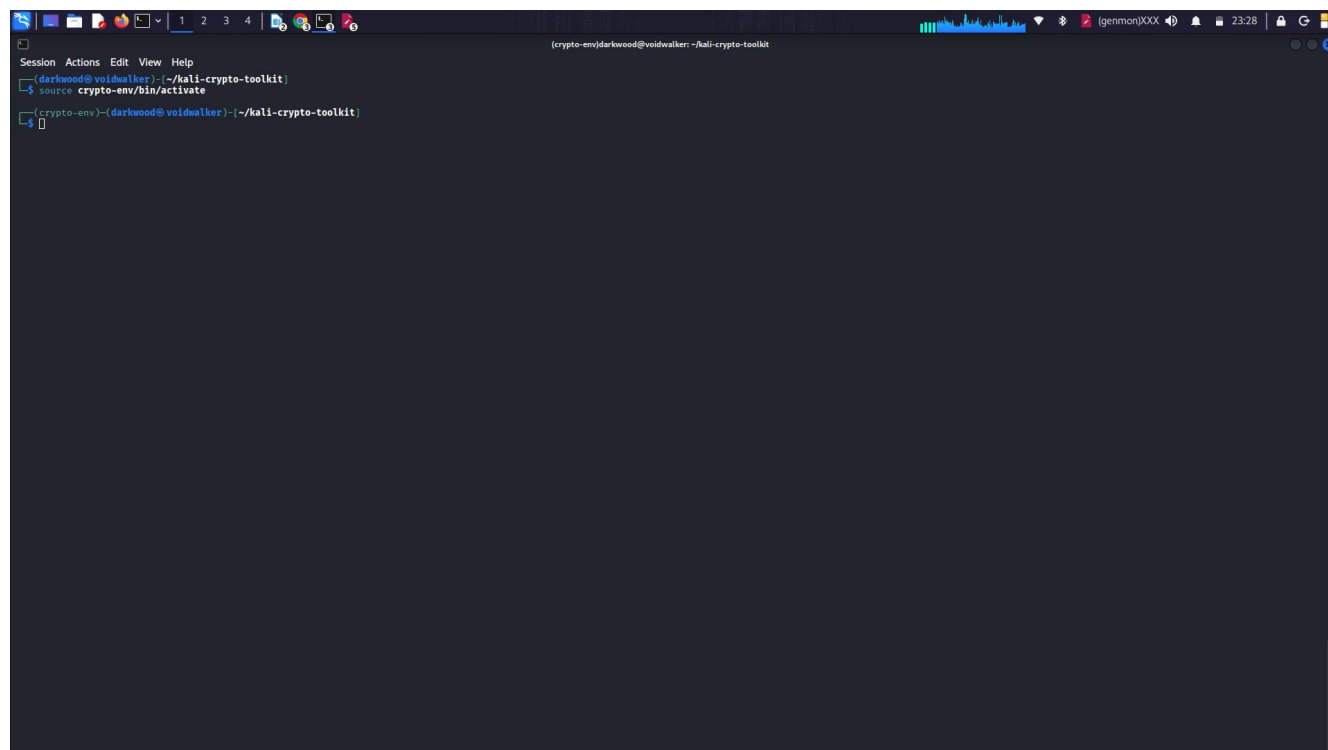
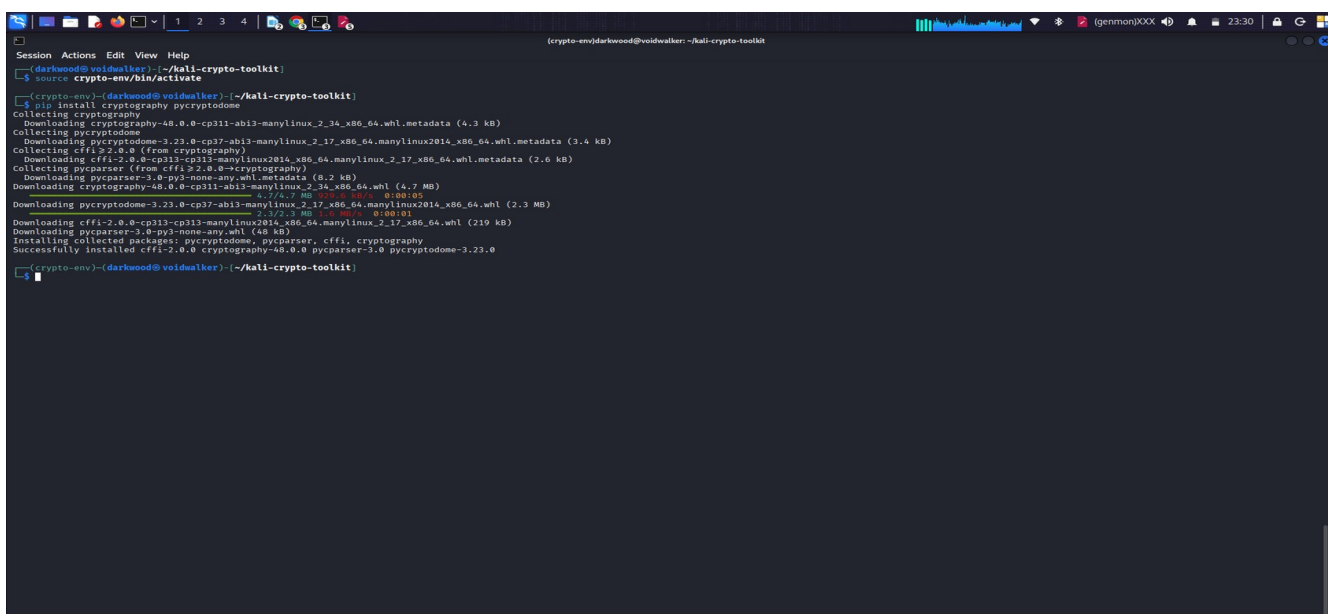


Fig 1.3: Activating environment (notice prompt change)

A terminal window with a dark background. The title bar shows '(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit'. The terminal content shows the user 'darkwood@voidwalker' in the directory '~/kali-crypto-toolkit' running the command 'source crypto-env/bin/activate'. The prompt changes from 'darkwood@voidwalker' to '(crypto-env)-(darkwood@voidwalker)'.

```
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit
Session Actions Edit View Help
darkwood@voidwalker: ~/kali-crypto-toolkit
$ source crypto-env/bin/activate
(crypto-env)-(darkwood@voidwalker): ~/kali-crypto-toolkit
$
```

Fig 1.4: Successful pip install

A terminal window showing the successful installation of cryptography and pycryptodome. The user runs 'pip install cryptography pycryptodome'. The output shows the collection and downloading of various wheel files for cryptography, pycryptodome, cffi, and pyparsing. The installation is successful.

```
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit
Session Actions Edit View Help
darkwood@voidwalker: ~/kali-crypto-toolkit
$ source crypto-env/bin/activate
(crypto-env)-(darkwood@voidwalker): ~/kali-crypto-toolkit
$ pip install cryptography pycryptodome
Collecting cryptography
  Downloading cryptography-48.0.0-cp311-abi3-manylinux_2_34_x86_64.whl.metadata (4.3 kB)
Collecting pycryptodome
  Downloading pycryptodome-3.22.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (3.4 kB)
Collecting cffi (from cryptography)
  Downloading cffi-2.0.0-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (2.0 kB)
Collecting pyparsing (from cffi>=2.0.0->cryptography)
  Downloading pyparsing-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloading cryptography-48.0.0-cp311-abi3-manylinux_2_34_x86_64.whl (4.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.7/4.7 MB 239.6 kB/s 0:00:05
Downloading pycryptodome-3.22.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.3/2.3 MB 1.0 MB/s 0:00:01
Downloading cffi-2.0.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (219 kB)
Downloading pyparsing-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pycryptodome, pyparsing, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-48.0.0 pyparsing-3.0 pycryptodome-3.22.0
(crypto-env)-(darkwood@voidwalker): ~/kali-crypto-toolkit
$
```

Fig 1.5: Complete environment test script

```
Session Actions Edit View Help
except:
    print(" * OpenSSL command failed")

# Summary
print("\n" + "-" * 70)
print("WEEK 1 SUMMARY")
print("-" * 70)
print(" * Python virtual environment configured")
print(" * Cryptography library installed (Fernet)")
print(" * PyCryptodome installed (AES)")
print(" * OpenSSL available for certificates")
print(" * Environment ready for Weeks 2-10")
print("-" * 70)
print("PROCEED TO WEEK 2: Classical Ciphers")
print("-" * 70)
EOF

# Run the Week 1 script
python3 ~/kali-crypto-toolkit/scripts/week1_complete.py

BIT4138: ADVANCED CRYPTOGRAPHY - WEEK 1
Cryptography Environment Setup Verification

Date/Time: 2026-06-07 23:51:35.116959
Python Versions: 3.13.12 (main, Feb 4 2026, 15:06:39) [GCC 15.2.0]
Platform: Linux 6.19.14+kali-amd64

[TEST 1] Fernet Symmetric Encryption (cryptography library)
 * Fernet key generated: b'ehP78301ti-1CTHvdxu' ...
 * Plaintext: b'BIT4138 Advanced Cryptography - Environment Ready'
 * Ciphertext: b'g4AAABQjJmRwZiCmQW993iccc' ...
 * Decrypted: b'BIT4138 Advanced Cryptography - Environment Ready'
 * VERIFICATION: Encryption/Decryption successful

[TEST 2] AES-256 Block Cipher (pycryptodome library)
 * AES-256 key generated: 00bfc808949e89d782a...
 * Nonce/IV: d8eb2fa7082f099b083...
 * Plaintext: b'AES-256 encryption test for BIT4138'
 * Ciphertext: 7ab7949bf3c5e9a54b8a736b8d57aafec62b9edfb2ab268b04ef493e6c...
 * Authentication tag: 4d69538c4914faa468e...
 * Decrypted: b'AES-256 encryption test for BIT4138'
 * VERIFICATION: AES-256 encryption/decryption successful

[TEST 3] OpenSSL System Check
 * OpenSSL: OpenSSL 3.6.2 7 Apr 2026 (Library: OpenSSL 3.6.2 7 Apr 2026)

WEEK 1 SUMMARY

 * Python virtual environment configured
 * Cryptography library installed (Fernet)
 * PyCryptodome installed (AES)
 * OpenSSL available for certificates
 * Environment ready for Weeks 2-10

PROCEED TO WEEK 2: Classical Ciphers

(crypto-env)-(darkwood@voidwalker)-(/kali-crypto-toolkit)
```

Week 2: Classical Cryptography and Cipher Design

Fig 2.1: Caesar Cipher implementation showing encryption of "CRYPTOGRAPHY"

```
Session Actions Edit View Help
(crypto-env)-(darkwood@voidwalker)-(/kali-crypto-toolkit)
$ >...
# Test multiple scenarios
test_caesar = [
    (ATTACK, 3, "Caesar"),
    (SECRET, 5, "Caesar"),
    (HELLO, "KEY", "Vigenere"),
    (CRYPTO, "PASS", "Vigenere")
]

print(" * Test Results Summary:")
print(" * Test Case | Cipher Type | Result")
print(" *-----")

# Caesar tests
caesar_test1 = CaesarCipher.encrypt("ATTACK", 3)
caesar_test1_dec = CaesarCipher.decrypt(caesar_test1, 3)
print(f" * ATTACK | Caesar | {'✓' if caesar_test1_dec == 'ATTACK' else '✗'}")

caesar_test2 = CaesarCipher.encrypt("secret", 5)
caesar_test2_dec = CaesarCipher.decrypt(caesar_test2, 5)
print(f" * secret | Caesar | {'✓' if caesar_test2_dec == 'secret' else '✗'}")

# Vigenere tests
vig_test1 = VigenereCipher.encrypt("HELLO", "KEY")
vig_test1_dec = VigenereCipher.decrypt(vig_test1, "KEY")
print(f" * HELLO | Vigenere | {'✓' if vig_test1_dec == 'HELLO' else '✗'}")

vig_test2 = VigenereCipher.encrypt("crypto", "PASS")
vig_test2_dec = VigenereCipher.decrypt(vig_test2, "PASS")
print(f" * crypto | Vigenere | {'✓' if vig_test2_dec == 'crypto' else '✗'}")

print("\n Security Weaknesses Identified:")
print(" 1. Caesar: Only 26 possible keys (can brute force in seconds)")
print(" 2. Caesar: Preserves letter frequencies (vulnerable to frequency analysis)")
print(" 3. Vigenere: Repeating keyword creates patterns")
print(" 4. Both: Not secure for modern applications")
print(" 5. Recommendation: Use AES or ChaCha20 for modern encryption")

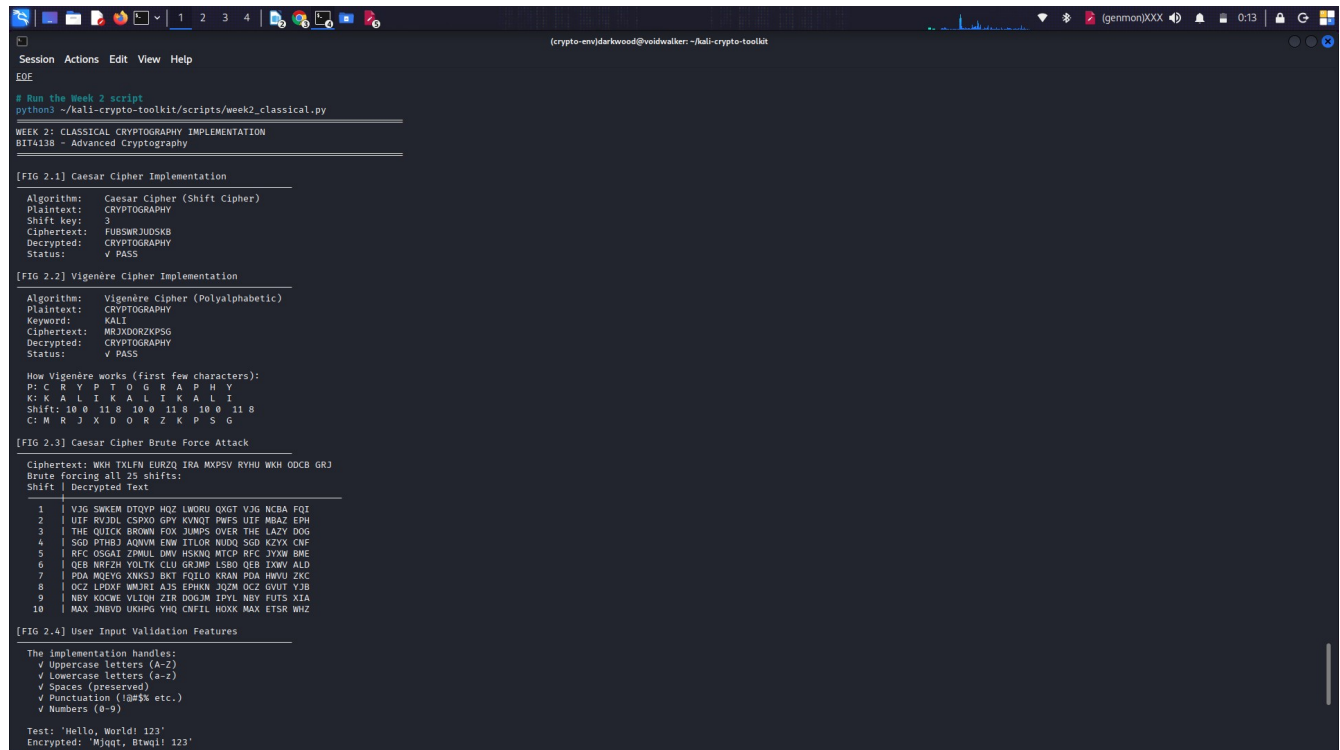
# Summary
print("\n" + "-" * 70)
print("WEEK 2 SUMMARY")
print("-" * 70)
print(" * Caesar Cipher implemented (shift cipher)")
print(" * Vigenere Cipher implemented (polyalphabetic)")
print(" * Brute force attack demonstrated (caesar weakness)")
print(" * Input validation preserves non-alphabetic characters")
print(" * Security analysis complete")
print("-" * 70)
print("READY FOR WEEK 3: Stream Ciphers (FSR & RC4)")
print("-" * 70)

# Demonstrate frequency analysis
demonstrate_frequency_weakness()

EOF

# Run the Week 2 script
```

Fig 2.2: Vigenère ("KALI"), Caesar Brute Force (Shift 3), and Input Validation



```
Session Actions Edit View Help
EOF
# Run the Week 2 script
python3 ~/kali-crypto-toolkit/scripts/week2_classical.py

WEEK 2: CLASSICAL CRYPTOGRAPHY IMPLEMENTATION
BIT4138 - Advanced Cryptography

[FIG 2.1] Caesar Cipher Implementation
Algorithm: Caesar Cipher (Shift Cipher)
Plaintext: CRYPTOGRAPHY
Shift key: 3
Ciphertext: FUBSWRJUDSKB
Decrypted: CRYPTOGRAPHY
Status: ✓ PASS

[FIG 2.2] Vigenère Cipher Implementation
Algorithm: Vigenère Cipher (Polyalphabetic)
Plaintext: CRYPTOGRAPHY
Keyword: KALI
Ciphertext: MRJXDRZKPSG
Decrypted: CRYPTOGRAPHY
Status: ✓ PASS

How Vigenère works (first few characters):
P: C R Y P T O G R A P H Y
K: K A L I K A L I K A L I
Shift: 10 0 11 8 10 0 11 8 10 0 11 8
C: M R J X D O R Z K P S G

[FIG 2.3] Caesar Cipher Brute Force Attack
Ciphertext: WKH TXLFN EURZQ IRA MXPSV RYHU WKH ODCB GRJ
Brute forcing all 25 shifts:
Shift | Decrypted Text
-----|-----
1 | VJG SWKEM BTQYP PHZ LWDUR QXGT VJG NGBA FRI
2 | UIF RVJDL CSPXO GPY KVNQT PMFS UIF MRAZ EPH
3 | THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG
4 | SGD PTMBJ AQWYM ENW ITLOR NUDQ SGD KZYX CNF
5 | RFC OSGAI ZPMUL DMV HSNQJ WTCF RFC JYXW DME
6 | QEB NRFZH YOLTX CLU GRJMP LSOB QEB IXWV ALD
7 | PDA MDEYG XNKSJ BKT FQILQ KRAN PDA HWVU ZKC
8 | OCZ LPDNF WHJRI AJG EPANW JQZM OCZ OVUT VJB
9 | NBV KOEWE VLIQH ZIR DQGM IPVL NBV FUTS XIA
10 | MAX JNBVD UKHPG YHQ CNFIL HOXX MAX ETSR WHZ

[FIG 2.4] User Input Validation Features
The implementation handles:
✓ Uppercase letters (A-Z)
✓ Lowercase letters (a-z)
✓ Spaces (preserved)
✓ Punctuation (like % etc.)
✓ Numbers (0-9)

Test: 'Hello, World! 123'
Encrypted: 'Wjqqt, Btwll! 123'
```

Fig 2.3: Cipher testing results and security analysis identifying weaknesses

```
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit

Session Actions Edit View Help
6 | QEB NRFZH YOLTK CLU GRJMP LSBO QEB IXWV ALD
7 | PDA MQEYV XNKSJ BKT QZILQ KXAN PDA HWU ZAC
8 | OCZ LPDXF MMJRI AJS EPHKN JQZM OCZ QVUT YJB
9 | NBY KCWE VLIQH ZIR DOGJM IPYL NBY FUTS XIA
10 | MAX JNBVD UKHPG YHQ CNFIL HOKX MAX ETSR WHZ

[FIG 2.4] User Input Validation Features

The implementation handles:
✓ Uppercase letters (A-Z)
✓ Lowercase letters (a-z)
✓ Spaces (preserved)
✓ Punctuation (!@#$% etc..)
✓ Numbers (0-9)

Test: 'Hello, World! 123'
Encrypted: 'Mjqqt, Btwqi! 123'
Non-letters preserved: ✓

[FIG 2.5] Cipher Testing Results & Security Analysis

Test Results Summary:
Test Case | Cipher Type | Result
ATTACK+3 | Caesar | ✓
secret+5 | Caesar | ✓
HELLO+KEY | Vigenere | ✓
crypto+PASS | Vigenere | ✗

Security Weaknesses Identified:
1. Caesar: Only 25 possible keys (can brute force in seconds)
2. Caesar: Preserves letter frequencies (vulnerable to frequency analysis)
3. Vigenere: Repeating keyword creates patterns
4. Both: Not secure for modern applications
5. Recommendation: Use AES or ChaCha20 for modern encryption

WEEK 2 SUMMARY
✓ Caesar Cipher implemented (shift cipher)
✓ Vigenere Cipher implemented (polyalphabetic)
✓ Brute force attack demonstrated Caesar weakness
✓ Input validation preserves non-alphabetic characters
✓ Security analysis completed

READY FOR WEEK 3: Stream Ciphers (LFSR & RC4)

FREQUENCY ANALYSIS DEMONSTRATION

Original: THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG
Encrypted: WKH TXLFN EURZQ IRA MXPSV RYHU WKH ODCB GRJ

In English, 'E' is most common letter (12.7%)
In ciphertext, 'W' appears most often
This reveals shift = 3 (EHW, but here EHW Actually EHW with shift 3)
Attackers can guess shift by finding most frequent letter!

(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
```

Week 3: Stream Ciphers and Randomness Testing

Fig 3: LFSR Implementation, RC4 Stream Cipher, and Performance Results

```
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit
Session Actions Edit View Help
print(f" Ciphertext (hex): {ciphertext.hex().hex()}")
print(f" Decrypted: {decrypted.decode()}")
print(f" Status: {'✓' if plaintext == decrypted.decode() else '✗'}")

# Performance
print("\n[Performance Test]")
test_data = b"x" * 10000
start = time.time()
RC4.encrypt(test_data, key)
elapsed = (time.time() - start) * 1000
print(f" Encrypted 10,000 bytes in {elapsed:.2f} ms")
print(f" Speed: {10000/elapsed:.1f} KB/s")

# XOR Property
print("\n[Stream Cipher Property]")
print(f" Encryption: Ciphertext = Plaintext ^ Keystream")
print(f" Decryption: Plaintext = Ciphertext ^ Keystream")
print(f" Same operation for both (XOR is symmetric)")

print("\n" + "-" * 70)
print("✓ LFSR implemented")
print("✓ RC4 working correctly")
print("✓ Performance tested")
print("-" * 70)
EOF

# Run the shortened script
python3 ~/kali-crypto-toolkit/scripts/week3_stream_short.py

=====
WEEK 3: STREAM CIPHERS - LFSR & RC4
=====

[LFSR - Linear Feedback Shift Register]
Polynomial: x^4 + x + 1
Seed: 1011 (binary)
Generated 30 bits: 01010101010101010101010101010101

[RC4 Stream Cipher]
Key: Kalikey
Plaintext: BIT4138 Stream Cipher Demo
Ciphertext (hex): b1b26817db06c8a142d0e937d92b79ea3c8da50623a95172c5fc ...
Decrypted: BIT4138 Stream Cipher Demo
Status: ✓

[Performance Test]
Encrypted 10,000 bytes in 5.22 ms
Speed: 1916.7 KB/s

[Stream Cipher Property]
Encryption: Ciphertext = Plaintext ^ Keystream
Decryption: Plaintext = Ciphertext ^ Keystream
Same operation for both (XOR is symmetric)

=====
✓ LFSR implemented
✓ RC4 working correctly
✓ Performance tested
=====

(crypto-env)-(darkwood@voidwalker): [~/kali-crypto-toolkit]
```

Week 4: AES Block Cipher

Fig 4: AES-256-CBC Key Generation, Encryption/Decryption, Performance Test, and Security Features

```
(crypto-env)-(darkwood@voidwalker): [~/kali-crypto-toolkit]
python3 ~/kali-crypto-toolkit/scripts/week4_aes_short.py

=====
WEEK 4: AES BLOCK CIPHER
=====

[AES Key Generation]
Algorithm: AES-256-CBC
Key size: 256 bits
Key (hex): 4014d22b4c2a2890e05aa887034f351bf38966f3 ...
Block size: 128 bits (16 bytes)
Mode: CBC (Cipher Block Chaining)

[Encryption & Decryption]
Plaintext: b'Confidential: Project launch date is 2026-12-01'
Ciphertext (hex): 2083c4d422badf8a0b0c1225fe9ca217a7fd7d30be0dd5454b2e6abff6d2506851249a15e73 ...
Ciphertext size: 64 bytes
Encryption time: 7.01 ms
Decrypted: b'Confidential: Project launch date is 2026-12-01'
Decryption time: 0.09 ms
Status: ✓ PASS

[File Encryption Demo]
Original size: 47 bytes
Encrypted size: 64 bytes
Overhead: 17 bytes (IV + padding)

[Performance Test]
Encrypted 100 KB in 0.46 ms
Speed: 218.0 MB/s

[Security Features]
✓ Random IV prevents identical plaintext patterns
✓ CBC mode chains blocks for better security
✓ PKCS7 padding handles non-16-byte data
✓ AES-256 provides 256-bit security strength

=====
WEEK 4 COMPLETE - Ready for Week 5: RSA
=====

(crypto-env)-(darkwood@voidwalker): [~/kali-crypto-toolkit]
```

Week 5: RSA Public Key Cryptography

Fig 5: RSA-2048 Key Pair Generation, Public Key Encryption, Private Key Decryption,

```
(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
$ python3 ~/kali-crypto-toolkit/scripts/week5_rsa_short.py

WEEK 5: RSA PUBLIC KEY CRYPTOGRAPHY

[RSA Key Pair Generation]
Algorithm: RSA
Key size: 2048 bits
Public exponent: 65537
Generation time: 23.89 ms
Public key (first 60 chars): -----BEGIN PUBLIC KEY-----
MIIBJANBgkqhkiG9w0BAQEFAAACAAQAA...

[Public Key Encryption]
Original: Secret meeting at 3pm in Lab 4B
Ciphertext (hex): 827e97ac4b1b3bef181489c0abe44f243c4e12628cc85dd995 ...
Ciphertext size: 256 bytes
Encryption time: 0.44 ms

[Private Key Decryption]
Decrypted: Secret meeting at 3pm in Lab 4B
Decryption time: 0.96 ms
Status: ✓ PASS

[Security & Comparison]
Asymmetric vs Symmetric:
- RSA uses two keys (public/private)
- AES uses one key (symmetric)
Max message size (2048-bit): 190 bytes with OAEP
Security level: ~112 bits (equivalent to AES-112)
Use case: Key exchange, digital signatures, small data

[Hybrid Encryption Note]
RSA is 100x slower than AES
Real systems: RSA for key exchange + AES for bulk data

(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
$
```

and Security Comparison

Week 6: Hashing and Password Security

Fig 6: SHA-256 Hash Generation, PBKDF2 Password Hashing with Salt, Login Authentication, and Security Analysis

```
(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
$ python3 ~/kali-crypto-toolkit/scripts/week6_hashing_short.py

WEEK 6: HASHING AND PASSWORD SECURITY

[SHA-256 Hash Function]
Message: B1f13B ADd0e0ad Cryptography
SHA-256: 0cb38ae2ab4649f48caf597ffca3f96c88661403fcd2dc2e3729eb39dd77f1c5
Hash length: 64 chars (256 bits)
Property: One-way - cannot reverse to get original

[Password Hashing with Salt - PBKDF2]
Password: MySecureP@ssw0rd123
Salt 1: 2bd29990c0b28c2848d0d2c45c79857a ...
Hash 1: 099b1ecce0a10184d4f6e98c2d8a0b ...
Salt 2: 17c3eb44d15b71c35838a1c2d439c7a1 ...
Hash 2: 7a8b85d0d172c2c7f624c4d0d0b3c36 ...
Same password, different hashes: ✓
Reason: Different random salts produce different hashes

[Login Authentication Simulation]
Login 'Correct_password123': ✓ ACCESS GRANTED
Login 'wrong_password': ✗ ACCESS DENIED

[Security Analysis]
Why hashing is irreversible:
- Cryptographic hash functions are one-way
- Small changes produce completely different hashes
- Avalanche effect: 1-bit change changes ~50% of hash bits
Importance of salt:
- Prevents rainbow table attacks
- Same password + different salts produce different hashes
PBKDF2 iterations (100,000):
- Slows down brute-force attacks
- Tunable for future hardware improvements

[Password Strength Test]
Weak password: password123
Hash: ef92b778baf271e89245b89ecbc0a4a4ae106c ...
Status: Easily cracked - dictionary attack
Strong password: X9#kL2!mQ7$wR4!
Hash: 7a0c4ccc7f132a03709849a9efc433440b0ab ...
Status: Secure - high entropy

(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
$
```

Creativity

```
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit/creative
Session Actions Edit View Help
(crypto-env)~(darkwood@voidwalker)-[~/kali-crypto-toolkit/creative]
$ ls
01_cover_banner.txt 02_crypto_timeline.txt 03_algorithm_comparison.html 04_crypto_cheatsheet.txt 05_certificate.txt 06_python_banner.py 07_creative_summary.txt create_all_creative.sh
(crypto-env)~(darkwood@voidwalker)-[~/kali-crypto-toolkit/creative]
$ cat ~/kali-crypto-toolkit/creative/05_certificate.txt

-----
CERTIFICATE OF COMPLETION
-----
BIT4138: Advanced Cryptography
Presented to: Gakere Samuel
Admission: BSCCS/2024/55278
Date: 2026

Completed Weeks 1-6

-----
[X] Environment Setup  [X] Classical Ciphers  [X] Stream Ciphers
[X] AES Block Cipher  [X] RSA Public Key  [X] Hashing & Pwd
-----
SIGNED
-----
Lecturer: MR NYORO MICHAEL
Date: 2026
-----

(crypto-env)~(darkwood@voidwalker)-[~/kali-crypto-toolkit/creative]
$ sudo npm install @hpress prisma @prisma/client bcrypt jsonwebtoken multer dotenv
```